



ConfigMap ve Secret

Kubernetes ve Konteyner Dünyası



Bu konuda

- ConfigMap ve Secret ayrımı
- Manifest ve üretim yolları
- Secret türleri ve base64 kodlaması
- Kullanım noktaları: env, envFrom ve volume
- Güncelleme sözleşmesi ve immutable
- Secret güvenliği ve özel registry

ConfigMap ve Secret nesneleri





ConfigMap ve Secret ayrımı

Soru	ConfigMap	Secret
Ne taşır?	Gizli olmayan yapılandırma	Hassas veri
Örnek	log seviyesi, özellik bayrağı	parola, token, TLS key
Pod spec nasıl tüketir?	env veya volume	env veya volume
Güvenlik varsayımı	Gizlilik sağlamaz	API nesnesidir; tek başına şifrelenmiş değildir

Pod'lar env ve volume referanslarını kullanarak başka namespace'lerdeki ConfigMap veya Secret nesnelere erişemez.



ConfigMap manifesti

ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: info
```

- ConfigMap `data` içinde UTF-8 metin, `binaryData` içinde base64 kodlu ikili veri taşır; aynı anahtar iki alanda birden bulunamaz.^[1]



Secret manifesti

Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  message: U2VsYW0hCg==
stringData:
  token: burada-bir-token-var
```

- Secret `data` içinde base64 kodlu değer tutar ama nesne oluşturma zamanında giriş kolaylığı için `stringData` kabul eder
- API `stringData` içeriğini `data` içine birleştirir; aynı anahtarda `stringData` kazanır^[2]
- Base64 kolayca geri çevrilebilir; gizlilik veya şifreleme sağlamaz^[5]
- Varsayılan kurulumda Secret verisi etcd içinde şifrelenmemiş olabilir^[2,5]



Oluşturma yolları

```
kubectl create configmap app-config --from-literal=LOG_LEVEL=info
kubectl create configmap app-files --from-file=app.yaml
kubectl create secret generic app-secret --from-file=token=./token.txt

kubectl create configmap app-config --from-literal=LOG_LEVEL=info \
  --dry-run=client -o yaml > configmap.yaml
kubectl apply -f configmap.yaml
kubectl get configmap app-config -o yaml
kubectl describe secret app-secret
```

- **Literal:** az sayıda test değeri için hızlıdır ^[8,9]
- **Dosya:** dosya adı varsayılan anahtar; `anahtar=dosya` ile açıkça seçilebilir ^[3,9]
- **Manifest:** gözden geçirilebilir ve tekrarlanabilirlik
- `describe secret` veri anahtarlarını gösterir, değerleri göstermez ^[11]



Secret türleri

type	Tipik kullanım	API'nin bildiği yapı
Opaque	Uygulamaya özel anahtar/değer	Özel biçim yok
kubernetes.io/dockerconfigjson	Özel registry kimliği	.dockerconfigjson
kubernetes.io/basic-auth	Kullanıcı/parola	username , password
kubernetes.io/ssh-auth	SSH özel anahtarı	ssh-privatekey
kubernetes.io/tls	TLS anahtarı ve sertifika	tls.key , tls.crt
kubernetes.io/service-account-token	Eski, uzun ömürlü ServiceAccount token'ı	Yeni kullanımda kısa ömürlü TokenRequest tercih edilir

Tablo kapsamlı değildir. Tür, veriyi otomatik olarak daha güvenli yapmaz; bazı yerleşik türlerde anahtar/biçim doğrulaması sağlar. ^[2]

Kullanım Şekilleri





Kullanım şekili seçimi

İhtiyaç	Pod alanı	Sonuç
Tek ConfigMap anahtarı	<code>configMapKeyRef</code>	Bir env değişkeni
Tek Secret anahtarı	<code>secretKeyRef</code>	Bir env değişkeni
Tüm anahtarlar	<code>envFrom</code>	Çok sayıda env değişkeni
Dosya/dizin	<code>configMap</code> veya <code>secret</code> volume	Her anahtar bir dosya
Aynı dizinde birden çok kaynak	<code>projected</code> volume	Birleştirilmiş dosya görünümü



Tek anahtar: **keyRef**

```
env:  
- name: LOG_LEVEL  
  valueFrom:  
    configMapKeyRef:  
      name: app-config  
      key: LOG_LEVEL  
- name: API_TOKEN  
  valueFrom:  
    secretKeyRef:  
      name: app-secret  
      key: token
```

- Pod spec, kaynak adı ile anahtarı açıkça bağlar
- Değer konteyner başlarken env içine enjekte edilir



Tüm anahtarlar: `envFrom`

```
envFrom:  
- configMapRef:  
  name: app-config  
- secretRef:  
  name: app-secret  
  optional: true  
prefix: SEC_
```

- Kaynaktaki her anahtar bir env adına dönüşür; `prefix` isteğe bağlıdır
- ConfigMap ve Secret varsayılan olarak zorunludur; `optional: true`, bulunmayan kaynağın yok sayılmasını sağlar ^[2,3,13]
- Aynı sonuç adı birden çok `envFrom` kaynağında varsa son kaynak kazanır; sabit `env` değeri ikisini de ezer ^[13]
- Çalışan konteynerin env değerleri nesne güncellendiğinde değişmez ^[1,2]



Volume olarak kullanım

```
volumeMounts:
- name: ayarlar
  mountPath: /etc/app
  readOnly: true
- name: kimlik
  mountPath: /var/run/app-secret
  readOnly: true
volumes:
- name: ayarlar
  configMap:
    name: app-config
- name: kimlik
  secret:
    secretName: app-secret
    defaultMode: 0400
```

- Her anahtar, mount dizininde bir dosya adına dönüşür^[1,4]
- `items` ile yalnızca gerekli anahtarlar ve hedef yollar seçilebilir^[1,4]
- Secret sadece ihtiyacı olan konteynere salt okunur bağlanır; uygulama içeriğini loglamamalıdır^[5]



projected Volume

```
volumes:  
- name: uygulama-parametreleri  
  projected:  
    defaultMode: 0400  
    sources:  
    - configMap:  
        name: app-config  
    - secret:  
        name: app-secret
```

Tek bir volume içine ConfigMap, Secret ve diğer desteklenen kaynaklar yansıtılabilir. Tüm kaynaklar Pod ile aynı namespace içinde bulunmalıdır. [\[6\]](#)

Kullanım ölçütü: Uygulama tek bir yapılandırma dizini bekliyorsa `projected`; kökenleri ayrı tutmak isteniyorsa ayrı volume'lar seçilir



subPath: tek dosya, farklı güncelleme sözleşmesi

```
volumeMounts:  
- name: ayarlar  
  mountPath: /etc/app/app.yaml  
  subPath: app.yaml
```

- Volume kökü yerine içindeki tek dosyayı hedef yola bağlar^[12]
- ConfigMap, Secret veya projected volume içindeki dosya `subPath` ile bağlanırsa otomatik güncelleme almaz^[1,2,6]
- Yeni değeri görmek için yeni Pod oluşturmak gerekir^[1,2]

Güncelleme ve Güvenlik





Güncelleme davranışları

Tüketim	Çalışan konteyner yeni değeri görür mü?	Operasyon
<code>env</code> , <code>envFrom</code> , <code>keyRef</code>	Hayır	Yeni Pod üret
Tam ConfigMap/Secret volume	Evet	Kubelet eşzamanlaması sonrası uygulama dosyayı yeniden okumalı
<code>projected</code> içinde ConfigMap/Secret	Evet	Aynı gecikme ve uygulama yeniden okuma koşulu geçerli
<code>subPath</code> dosyası	Hayır	Yeni Pod üret



Değişiklik, rollout ve kimlik döndürme

ConfigMap veya Secret nesnesini güncellemek Pod template'ini değiştirmez. Bu yüzden env ve `subPath` tüketiminde yeni Pod üretimi ayrıca planlanır.^[1,2]

Kaynak sözleşmelerinden türetilen operasyon örneği:

1. Yeni adı olan bir Secret üretilir: örneğin `db-credentials-v2`
2. Workload'un Pod template referansı yeni ada taşınır
3. Yeni Pod'ların sağlıklı olduğu ve yeni kimliği kullandığı doğrulanır
4. Eski kimlik kaynak sistemde iptal edilir; sonra eski Secret silinir



Immutable

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-v1
immutable: true
data:
  LOG_LEVEL: info
```

- `immutable: true`, ConfigMap ve Secret için geri alınamaz ^[1,2]
- ConfigMap'ın `data` / `binaryData` alanları ile Secret'ın `data` içeriği artık değiştirilemez ^[1,2]
- Değişiklikte yeni adla nesne üretilir, tüketici taşınır, sonra eski nesne silinir
- Mevcut volume bağıını yeni nesneye geçirmek için Pod yeniden oluşturulur ^[1,2]



Secret için savunma katmanları

- Cluster tarafında Secret verisi için etcd'de **saklama zamanında şifreleme** etkinleştirilir^[5]
- Erişim en az ayrıcalıkla sınırlandırılır; `list` yetkisi bile içerikleri dolaylı açığa çıkarabilir^[5]
- Secret kullanan Pod oluşturabilen kişi, doğrudan Secret okuma izni olmasa da değeri açığa çıkarabilir^[5]
- Secret yalnızca ihtiyacı olan konteynere bağlanır^[5]
- Uygulamanın gizli değeri loglaması veya hata mesajına koyması engellenir^[5]
- Kısa ömürlü kimlik tercih edilir; düzenli yenileme ve eski kimliği iptal etme, yukarıdaki kaynak sözleşmelerinden türetilen işletim örneğidir^[2,5]
- Canlı değer imaja, manifest deposuna veya komut geçmişine konmaz^[2,5,7]



Özel registry

```
kubectl -n uygulama create secret docker-registry regcred \  
--from-file=./config.json
```

```
spec:  
  imagePullSecrets:  
    - name: regcred
```

- Komut, `kubernetes.io/dockerconfigjson` türünde kimlik üretir^[10]
- `regcred` ve Pod aynı `uygulama` namespace'i içinde olmalıdır^[2,7]
- Kullanıcı/parola bayrakları terminal geçmişi veya süreç listesinde görünebilir; mümkünse güvenli hazırlanmış Docker config dosyası kullanılır^[7]
- `imagePullSecrets` alanı burada doğru Secret türüyle uygulanır^[2]

Kaynaklar





Kaynaklar

1. ConfigMaps: kubernetes.io/docs/concepts/configuration/configmap/
2. Secrets: kubernetes.io/docs/concepts/configuration/secret/
3. Configure a Pod to Use a ConfigMap: kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/
4. Distribute Credentials Securely Using Secrets: kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/
5. Good Practices for Kubernetes Secrets: kubernetes.io/docs/concepts/security/secrets-good-practices/
6. Projected Volumes: kubernetes.io/docs/concepts/storage/projected-volumes/
7. Pull an Image from a Private Registry: kubernetes.io/docs/tasks/configure-pod-container/pull-image-private-registry/
8. `kubectl create configmap`: kubernetes.io/docs/reference/kubectl/generated/kubectl_create/kubectl_create_configmap/
9. `kubectl create secret generic`:
kubernetes.io/docs/reference/kubectl/generated/kubectl_create/kubectl_create_secret_generic/
10. `kubectl create secret docker-registry`:
kubernetes.io/docs/reference/kubectl/generated/kubectl_create/kubectl_create_secret_docker-registry/
11. Manage Secrets Using kubectl: kubernetes.io/docs/tasks/configmap-secret/managing-secret-using-kubectl/
12. Volumes: kubernetes.io/docs/concepts/storage/volumes/
13. Pod v1 API: kubernetes.io/docs/reference/kubernetes-api/core/pod-v1/